pyflic Usage Guide

Source code: https://github.com/PletcherLab/pyflic

Table of Contents

1. What pyflic does
2. Project directory layout
3. How licks and events are calculated
4. Configuration file (`flic_config.yaml`)
5. Scripts -- automated hub pipelines (`scripts:`)
6. Command-line tools
7. Graphical tools
8. Python API
9. Advanced analytics
10. Performance and caching
11. Typical workflows
12. Jupyter notebooks

1. What pyflic does

pyflic analyzes data from **FLIC (Fly Liquid-food Interaction Counter)** experiments, which measure licking behavior in fruit flies using electrical signal data. It detects feeding and tasting bouts, generates quality-control reports, and produces publication-ready plots, summary tables, and statistical comparisons.

The main objects in pyflic map to the physical hardware:

- A **DFM** (Data File Module) is one physical FLIC device, reading up to 12 wells.
- A **chamber** is a group of wells (1 or 2) within a DFM assigned to one experimental treatment.
- An **experiment** is a collection of DFMs governed by a shared configuration.

2. Project directory layout

pyflic expects one directory per experiment with the following structure:

```
project_dir/
  flic_config.yaml        <-- required; defines experiment structure and
parameters
  data/                   <-- DFM CSV files (one per DFM, named by device
ID)
  remove_chambers.csv     <-- optional; named exclusion groups (see Section
4)
  flic_config_results/    <-- outputs for flic_config.yaml (auto-created)
    qc/                   <-- QC output (or qc_<start>_<end> for ranged
loads)
```

```
    analysis/              <-- analysis output (or analysis_<start>_<end>)
  my_other_config_results/ <-- outputs for my_other_config.yaml (same
pattern)
  .pyflic_cache/           <-- auto-managed disk cache for feeding summaries
(safe to delete)
```

Data files must live in the `data/` subdirectory and follow one of these naming conventions:

- **v3 format:** `DFM{id}_{segment}.csv` (e.g. `DFM1_0.csv`, `DFM1_1.csv`)
- **v2 format:** `DFM_{id}.csv` or `DFM_{id}_{segment}.csv`

Multi-segment experiments are stitched together automatically in filename order.

> Output directories are namespaced by config filename stem. For example, loading `my_protocol.yaml` writes to `project_dir/my_protocol_results/`.

---

3. How licks and events are calculated

When a DFM is loaded, pyflic runs the following signal processing pipeline automatically for every well. Understanding this pipeline is essential for choosing good parameter values and interpreting the output correctly.

Step 1: Load raw data

Raw CSV files are read and concatenated if the experiment spans multiple segments. A `Minutes` column is computed from the elapsed time (using `Date`/`Time`/`MSec` columns or a pre-computed `Seconds` column).

Step 2: Baseline subtraction

A **running median** is applied to each well's raw signal with a centered window defined by `baseline_window_minutes` (default 3 minutes, which equals 900 samples at 5 Hz). The baseline assumption is that feeding interactions in any given 3-minute window are sufficiently rare that the median of the signal over that window represents the background. The **baselined signal** is the raw signal minus this running median.

This step corrects for:

- Inter-DFM signal variation (different hardware, different food levels)
- Slow signal drifts over time (e.g. food evaporation, well depletion)

Step 3: Compute per-well thresholds

For each well, threshold values are derived from `feeding_threshold`, `feeding_minimum`, `tasting_minimum`, and `tasting_maximum`. When `feeding_threshold` is negative, it is interpreted as a fraction of the well's maximum baselined signal (adaptive thresholding). When positive, thresholds are fixed constants.

Step 4: Identify feeding licks and events

This is the core detection pipeline, applied independently to each well:

1. **Threshold crossing.** Each sample is compared against two thresholds:
    - `feeding_minimum` (the lower threshold) -- marks a sample as a *candidate lick*
    - `feeding_threshold` (the upper threshold) -- marks a sample as a *confirmed lick*
2. **Surviving events.** Contiguous runs of candidate licks are identified. A run is retained as a **surviving event** only if at least one sample within it also crosses the upper `feeding_threshold`. This two-threshold approach prevents weak noise from being counted as feeding while still capturing the full duration of real feeding bouts (which may dip below `feeding_threshold` momentarily mid-bout).
3. **Minimum event length.** Surviving events shorter than `feeding_minevents` samples are discarded. At 5 Hz, the default of 1 means even single-sample contacts are kept.
4. **Event linking (the link gap).** After the initial event detection, short gaps *between* events are bridged. If two events are separated by a FALSE (non-lick) run of `feeding_event_link_gap` or fewer samples, the gap is filled in and the two events merge into one. At the default of 5 samples (1 second at 5 Hz), brief interruptions mid-meal are ignored. Gaps at the very start or end of the recording are never bridged -- only interior gaps between two existing events are linked.
5. **Final event extraction.** After linking, the merged boolean lick vector is scanned to produce the final event vector: each event is recorded as its start position and duration (in samples).

The link gap parameter has a substantial effect on the number and duration of detected events. A larger gap merges more events, producing fewer but longer bouts. A smaller gap preserves short interruptions as separate events.

Step 5: Well A / Well B assignment (two-well experiments)

The `pi_direction` parameter maps physical well positions to logical Well A / Well B labels. `pi_direction="left"` makes the left well (odd-numbered: W1, W3, W5, ...) Well A. `pi_direction="right"` reverses this, making the right well (even-numbered: W2, W4, W6, ...) Well A.

Swapping `pi_direction` across DFMs while counterbalancing food positions ensures that a positive PI always indicates preference for Well A regardless of its physical location.

Step 6: Dual-feeding correction (two-well, optional)

When `correct_for_dual_feeding` is enabled, pyflic detects samples where *both* wells in a chamber register feeding licks simultaneously. The baseline of the non-preferred well is adjusted to reduce the signal contributed by crosstalk, and the feeding detection pipeline is re-run on the corrected signal.

Step 7: Identify tasting licks

Signals that fall between `tasting_minimum` and `tasting_maximum` and were *not* already classified as feeding licks are labelled tasting licks. Contiguous runs are grouped into tasting events subject to `tasting_minevents`.

Step 8: Compute PI (two-well experiments)

The **Preference Index (PI)** is computed as:

```
PI = (LicksA - LicksB) / (LicksA + LicksB)
```

PI ranges from -1 (strong preference for Well B) to +1 (strong preference for Well A). An **Event PI** is computed similarly using event counts instead of lick counts.

Step 9: Compute durations and intervals

For each feeding event, the following metrics are computed:

- **Duration** (seconds): event length / `samples_per_second`
- **Total intensity**: sum of baselined signal values within the event
- **Average intensity**: mean signal within the event
- **Max / min intensity**: peak and trough signal within the event

Inter-event intervals (the time between consecutive events) are also computed and stored.

Step 10: Light state

If the DFM CSV contains `OptoCol1` (and optionally `OptoCol2`) columns, the per-well light-on/off state is decoded from the bit-encoded values and stored in `lights_df`. This is used for light-on markers on cumulative lick plots and for the light-phase summary analysis.

---

4. Configuration file (`flic_config.yaml`)

The YAML config is the entry point for every experiment. It defines the experiment type, algorithm parameters, and how chambers are assigned to treatments. The easiest way to create one is with the `pyflic config` GUI (see Command-line tools).

You can validate a config file at any time:

```
pyflic lint /path/to/project_dir
```

The linter reports errors and warnings with line numbers when possible.

Top-level structure

```
global:
  experiment_type: ...
  constants: { ... }
  params: { ... }
  experimental_design_factors: { ... }   # optional
  well_names: { ... }                    # optional

dfms:
  1:
    params: { ... }
```

```
      chambers:
        1: TreatmentA
        2: TreatmentB
    2:
      ...

  scripts:                                # optional -- see Section 5
    - name: "My Pipeline"
      steps:
        - action: load
        - action: basic_analysis
```

> **Note:** The `data_dir` key is no longer used. Data is always read from `project_dir/data/`. If an old config contains `data_dir`, the linter will flag it for removal.

`global.experiment_type`

Determines which analysis class is loaded. If omitted, pyflic selects automatically based on `chamber_size`.

Value	Class	Use when
`single_well`	`SingleWellExperiment`	12 independent wells per DFM
`two_well`	`TwoWellExperiment`	Two-well choice experiments
`hedonic`	`HedonicFeedingExperiment`	Two-well with hedonic/reward-based analysis
`progressive_ratio`	`ProgressiveRatioExperiment`	Progressive-ratio schedules

`global.constants`

Hard limits used during data validation. Chambers that exceed these thresholds are flagged or excluded.

Key	Default	Description
`min_untransformed_licks_cutoff`	*(unset)*	Minimum non-transformed lick count; chambers where at least one well is below this are excluded by `auto_remove_chambers()`
`max_med_duration_cutoff`	`13`	Maximum median bout duration (seconds); chambers above are flagged
`max_events_cutoff`	`150000`	Maximum number of raw events; chambers above are flagged

---

`global.params`

Algorithm parameters used for bout detection. These are global defaults; any DFM-level `params` block overrides them for that device only. See Section 3 for how each parameter affects the detection pipeline.

Timing

Key	Default	Description
`baseline_window_minutes`	`3`	Duration (minutes) of the rolling median window for baseline subtraction
`samples_per_second`	`5`	Hardware sampling rate in Hz

Feeding detection

Key	Default	Description
`feeding_threshold`	`10`	Upper threshold: at least one sample in a candidate event must exceed this to be retained
`feeding_minimum`	`10`	Lower threshold: samples above this are candidate licks
`feeding_minevents`	`1`	Minimum event length in samples; shorter events are discarded
`feeding_event_link_gap`	`5`	Maximum gap (in samples) between events that are bridged into one bout (~1 s at 5 Hz)

Tasting detection

Key	Default	Description
`tasting_minimum`	`0`	Lower threshold for tasting licks
`tasting_maximum`	`10`	Upper threshold; contacts above this are feeding, not tasting
`tasting_minevents`	`1`	Minimum samples per tasting event

Hardware / experiment design

Key	Required	Description
`chamber_size`	Yes	Wells per chamber: `1` (single-well) or `2` (two-well choice)
`pi_direction`	No	Which side is the preference-index reference: `"left"` or `"right"`
`correct_for_dual_feeding`	No	If `true`, corrects simultaneous licks in two-well experiments

> Many parameter names have accepted aliases (e.g. `baseline_window` is equivalent to `baseline_window_minutes`, `link_gap` to `feeding_event_link_gap`). The config editor handles these automatically.

`global.experimental_design_factors` (optional)

Defines a factorial experimental design. Each key is a factor name; its value is the list of levels. When factors are defined, chamber assignments in `dfms` must be written as comma-separated factor levels in the same order as the factors are listed here.

```
experimental_design_factors:
  paired: [Paired, Unpaired]
  genotype: [Chrim, WCS]
```

With this definition a chamber assigned `"Paired,Chrim"` will be tagged with `paired=Paired` and `genotype=Chrim`.

`global.well_names` (optional)

Human-readable labels for the wells within each chamber, used in plots and reports.

```
# Two-well (chamber_size: 2)
well_names:
  A: Sucrose
  B: Yeast
```

`dfms`

One entry per physical DFM device. The key (or `id` field) matches the device number used in the CSV filenames.

```
dfms:
  1:
    params:
      pi_direction: left      # override global param for this DFM only
    chambers:
      1: Paired,WCS           # chamber 1: Paired genotype WCS
      2: Unpaired,WCS         # chamber 2: Unpaired genotype WCS
  2:
    params: {}                # no overrides; inherits all global params
    chambers:
      1: Unpaired,Chrim
      2: Paired,Chrim
```

- The `params` block accepts the same keys as `global.params` and takes precedence over global defaults for that DFM.
- The `chambers` block maps chamber index to treatment name (or comma-separated factor levels if `experimental_design_factors` is defined).
- `excluded_chambers` in YAML is deprecated and ignored at load time. Use `remove_chambers.csv` groups instead (below).

`remove_chambers.csv` (recommended exclusion workflow)

Chamber exclusions are stored in `project_dir/remove_chambers.csv`, not in `flic_config.yaml`.

CSV format:

```
group,dfm_id,chamber,note
general,1,3,low lick count
general,2,5,
Standard Analysis,1,4,noisy signal
```

- `group` lets you keep multiple exclusion sets in one file.
- `load_experiment_yaml(..., exclusion_group="general")` applies one group at load.
- The hub **Remove chambers** button applies group `general`.
- Script action `remove_chambers` defaults to the script's `name` as the group when `group:` is omitted.
- QC Viewer can save current exclusions to any named group via **Save removed chambers...**.

Complete example

```yaml
global:
  experiment_type: hedonic
  constants:
    min_untransformed_licks_cutoff: 20
    max_med_duration_cutoff: 13
    max_events_cutoff: 150000
  params:
    chamber_size: 2
    pi_direction: left
    baseline_window_minutes: 3
    samples_per_second: 5
    feeding_threshold: 10
    feeding_minimum: 10
    feeding_minevents: 1
    feeding_event_link_gap: 5
    tasting_minimum: 0
    tasting_maximum: 10
    tasting_minevents: 1
    correct_for_dual_feeding: true
  experimental_design_factors:
    paired: [Paired, Unpaired]
    genotype: [Chrim, WCS]
  well_names:
    A: Sucrose
    B: Yeast

dfms:
  1:
    params:
      pi_direction: left
```

```
      chambers:
        1: Paired,WCS
        2: Unpaired,WCS
    2:
      params:
        pi_direction: right
      chambers:
        1: Unpaired,Chrim
        2: Paired,Chrim
```

---

5. Scripts -- automated hub pipelines (`scripts:`)

The optional `scripts:` key in `flic_config.yaml` lets you define named analysis pipelines that run in a single click from the **pyflic hub** GUI. When at least one script is defined, a dropdown and **Run Script** button appear in the Load group of the hub.

Structure

```
scripts:
  - name: "Standard Analysis"
    start: 0
    end: 0
    steps:
      - action: load
      - action: basic_analysis
      - action: feeding_csv
      - action: binned_csv
        binsize: 30
      - action: plot_feeding_summary
      - action: plot_binned
        metric: Licks
        mode: total
        binsize: 30
      - action: plot_dot
        metric: PI
      - action: pdf_report
```

Each script is a named list of **steps**. Every step has an `action:` key plus optional parameters. Steps execute in order; they all share the experiment loaded by the first `load` step (or auto-loaded on first use if no `load` step appears).

Parameter resolution order

For each step, parameter values are resolved in this order:

1. **Per-step value** -- set directly on the step (e.g. `binsize: 30`)
2. **Script-level default** -- `start:` / `end:` set at the script level

3. **UI value** -- the spinbox / control value currently shown in the hub

---

Supported actions

Core actions

`action:`	Parameters	Experiment type
`load`	`start`, `end`, `parallel`	all
`remove_chambers`	`group`	all
`write_summary`	`start`, `end`	all
`basic_analysis`	`start`, `end`	all
`feeding_csv`	`start`, `end`	all
`binned_csv`	`start`, `end`, `binsize`	all
`weighted_duration`	`start`, `end`	hedonic only
`plot_feeding_summary`	`start`, `end`	all
`plot_binned`	`metric`, `mode`, `binsize`, `start`, `end`	all
`plot_dot`	`metric`, `mode`, `start`, `end`	all
`plot_well_comparison`	`metric`, `start`, `end`	two-well only
`plot_hedonic`	`start`, `end`	hedonic only
`plot_breaking_point`	`config` (1--4), `start`, `end`	progressive_ratio only

Advanced analytics actions

`action:`	Parameters	Description
`tidy_export`	`kind` (`feeding` or `tasting`)	Write a long-format CSV with one row per bout
`bootstrap`	`metric`, `mode`, `n_boot`, `ci`, `seed`	Write bootstrap confidence intervals per treatment
`compare`	`metric`, `mode`, `model` (`aov` or `lmm`), `factors`	Run ANOVA or linear mixed model with optional Tukey HSD posthoc
`light_phase_summary`		Write per-chamber feeding stats split by light/dark phase
`param_sensitivity`	`parameter`, `values`	Sweep a parameter and report Licks, Events, MedDuration per treatment
`transition_matrix`		Write A/B bout transition counts per chamber (two-well only)

action:	Parameters	Description
pdf_report	metrics, binsize	Generate a PDF combining summary, plots, and statistics

Actions that require a specific experiment type are skipped with a log message rather than raising an error.

---

plot_binned and plot_dot -- metric and mode

metric names for **two-well** experiments: Licks, PI, EventPI, LicksA, LicksB, Events, MedDuration, MedDurationA, MedDurationB, MeanDuration, MedTimeBtw

metric names for **single-well** experiments: Licks, Events, MedDuration, MeanDuration, MedTimeBtw, MeanInt, MedianInt

mode is optional. When omitted, a sensible default is chosen automatically. Valid mode values: total, mean_ab, A, B

Example with advanced analytics

```yaml
scripts:
  - name: "Full Pipeline with Stats"
    steps:
      - action: load
      - action: basic_analysis
      - action: feeding_csv
      - action: tidy_export
      - action: compare
        metric: MedDuration
        model: aov
      - action: bootstrap
        metric: PI
        n_boot: 5000
        ci: 0.95
      - action: light_phase_summary
      - action: param_sensitivity
        parameter: feeding_event_link_gap
        values: [0, 2, 5, 10, 15, 20]
      - action: transition_matrix
      - action: pdf_report
```

---

6. Command-line tools

pyflic installs a unified pyflic command with subcommands, plus standalone entry points for backward compatibility.

Command	Standalone	Description

Command	Standalone	Description
`pyflic config`	`pyflic-config`	Launch the config editor GUI
`pyflic qc <project>`	`pyflic-qc <project>`	Launch the QC viewer
`pyflic hub [project]`	`pyflic-hub [project]`	Launch the analysis hub GUI
`pyflic lint <path>`	`pyflic-lint <path>`	Validate a `flic_config.yaml` against the schema
`pyflic report <project>`		Generate a PDF experiment report
`pyflic clear-cache <project>`		Remove disk-cached feeding summaries
`pyflic version`		Print the installed version

`pyflic lint`

Schema-validates `flic_config.yaml` and reports issues with line numbers when possible. Checks for unknown keys, invalid `pi_direction` values, mismatched factor levels, missing `chamber_size`, duplicate DFM IDs, and deprecated `data_dir` usage.

```
pyflic lint /path/to/project
# output:
# flic_config.yaml:12 error: DFM 3: pi_direction must be 'left' or 'right',
got 'sideways'
# 1 error(s), 0 warning(s)
```

`pyflic report`

Generates a multi-page PDF report at `project_dir/<config_stem>_results/analysis/experiment_report.pdf` containing the experiment summary, a feeding summary table (key metrics only), binned time-course plots for Licks, Events, and MedDuration, and an ANOVA comparison with Tukey HSD posthoc.

```
pyflic report /path/to/project
```

When using the default config (`flic_config.yaml`), the report is written under `project_dir/flic_config_results/analysis/`.

`pyflic clear-cache`

Removes all files in `.pyflic_cache/` for a project. Safe to run at any time -- the cache is automatically rebuilt on the next load.

```
pyflic clear-cache /path/to/project
```

---

7. Graphical tools

Config Editor (`pyflic config`)

Creates and edits `flic_config.yaml` without hand-editing YAML. The editor provides:

- **Experiment settings** -- chamber size, experiment type, well names
- **Global parameters** -- all detection parameters with labeled fields
- **Experimental design factors** -- define factorial designs with factor names and levels
- **Per-DFM tabs** -- parameter overrides, chamber-to-treatment assignments, chamber exclusions
- **Constants** -- QC cutoff values

Automatically loads `flic_config.yaml` from the current directory on startup. Saves directly to YAML.

Analysis Hub (`pyflic hub`)

The primary GUI for running analyses. The hub is organized as cards (Project, Load, Analyze, Plots, Scripts, Tools):

Project card:

- Project folder + config selector (`*.yaml` in the directory)
- **Run action for every YAML config** toggle (batch mode)
- **YAML info...** popup summarizing type/chamber size/scripts/exclusions per YAML

Load card:

- Time range, parallel-load toggle, and bin size
- **Load experiment**
- **Remove chambers** (applies `general` group from `remove_chambers.csv`)
- Launchers for Config Editor and QC Viewer

Scripts card:

- Script dropdown (single-yaml mode) or union of script names (batch mode)
- **Run Script**
- **Run All Scripts** (all scripts in the active YAML)

Analyze card -- core:

- Run full basic analysis
- Write feeding summary CSV
- Write binned feeding summary CSV
- Write weighted duration summary (hedonic only)

Analyze card -- advanced:

- **Tidy events CSV** -- one row per bout, long format suitable for downstream tools
- **Bootstrap CIs** -- prompts for a metric name and iteration count, then writes per-treatment bootstrap confidence intervals
- **Compare treatments (ANOVA / LMM)** -- prompts for a metric and model type, runs the statistical test, writes results and posthoc tables
- **Light-phase summary CSV** -- per-chamber feeding stats split by light vs dark phase using OptoCol1
- **Parameter sensitivity sweep** -- prompts for a parameter and values to sweep, then writes Licks/Events/MedDuration mean and SEM per treatment for each value
- **Bout transition matrix** -- A-to-A, A-to-B, B-to-A, B-to-B counts per chamber (two-well only)
- **Write PDF report** -- generates the full experiment report

Tools card:

- **Lint config** -- validate the currently selected YAML without leaving the hub
- **Compare two configs** -- select a second project directory and compare per-treatment metric means
- **Clear disk cache** -- remove `.pyflic_cache/` for the current project

Plots card:

- Feeding summary plot
- Binned time-course with metric/mode selector
- Dot plot with metric/mode selector
- Well A vs B comparison (two-well only)
- Hedonic feeding plot (hedonic only)
- Breaking-point plots (progressive ratio only)

All outputs (CSVs, PNGs, PDFs) are written to `project_dir/<config_stem>_results/analysis[_start_end]/`.

QC Viewer (`pyflic qc`)

Interactive dashboard for inspecting QC results and managing chamber exclusions.

Load tab:

- Project directory, time range, and parallelism options
- Load, Edit Config, and Reload Config buttons

Feeding Summary tab:

- Table of all chambers with per-chamber metrics
- Exclusion checkboxes per chamber (bidirectionally synced with DFM tabs)
- Auto-filter button (applies `constants`-based cutoffs)

Params tab (live recompute):

- Spinboxes for `baseline_window_minutes`, `feeding_threshold`, `feeding_minimum`, `tasting_minimum`, `tasting_maximum`, `feeding_event_link_gap`, and `feeding_minevents`
- **Recompute** button that re-runs the feeding/tasting detection pipeline on all loaded DFMs without re-reading data files, then refreshes the Feeding Summary tab
- Changes are *not* written to `flic_config.yaml` -- this is for interactive exploration only

DFM tabs (one per loaded DFM):

- Sub-tabs: Integrity, Data Breaks, Simultaneous Feeding, Bleeding, Raw Signal, Baselined Signal, Cumulative Licks
- Per-well exclusion checkboxes (synced with Feeding Summary)

Exclusions are managed in-memory while reviewing, and can be persisted to `remove_chambers.csv` via **Save removed chambers...** under a named group.

---

8. Python API

Loading an experiment

```
from pyflic import load_experiment_yaml

exp = load_experiment_yaml(
    "/path/to/project_dir",
    config_name="flic_config.yaml", # select a specific YAML in project_dir
    range_minutes=(0, 0),       # (start, end); (0, 0) = full recording
    parallel=True,              # load DFMs concurrently
    eager=True,                 # pre-compute feeding summary on load (set
False to skip)
    use_disk_cache=True,        # read/write .pyflic_cache/ for feeding
summaries
    exclusion_group="general",  # apply one remove_chambers.csv group; None
disables file exclusions
)
```

`load_experiment_yaml` reads `flic_config.yaml` and returns the appropriate subclass (`SingleWellExperiment`, `TwoWellExperiment`, `HedonicFeedingExperiment`, or `ProgressiveRatioExperiment`).

Experiment class hierarchy

```
Experiment (base)
+-- SingleWellExperiment        # chamber_size=1; 12 independent wells per
DFM
+-- TwoWellExperiment           # chamber_size=2; two-well choice
    +-- HedonicFeedingExperiment
    +-- ProgressiveRatioExperiment
```

All subclasses share the same core API. Specialized methods (breakpoint analysis, hedonic metrics) are added in the subclasses.

Key methods

Accessing data

```
exp.dfms                        # dict of DFM objects keyed by DFM ID
dfm = exp.get_dfm(1)            # get a single DFM
exp.design                      # ExperimentDesign (structure, treatments,
factor levels)
```

Running QC

```
exp.write_qc_reports()          # write to project_dir/qc/
results = exp.compute_qc_results()  # get results as Python dicts
```

Feeding summary

```
df = exp.feeding_summary()
df = exp.feeding_summary(range_minutes=(30, 90))
binned = exp.binned_feeding_summary(binsize_min=30)
```

Plotting

```
fig = exp.plot_feeding_summary()
fig = exp.plot_binned_metric_by_treatment(metric="Licks", binsize_min=30)
fig = exp.plot_dot_metric_by_treatment(metric="MedDuration")
fig = exp.plot_cumulative_licks_chamber(dfm_id=1, chamber=1)
```

Full pipeline

```
results = exp.execute_basic_analysis()
```

Text summary and PDF report

```
print(exp.summary_text())
exp.write_summary()

from pyflic import write_experiment_report
write_experiment_report(exp)    # writes to
project_dir/<config_stem>_results/analysis/experiment_report.pdf
```

DFM objects

```
dfm = exp.get_dfm(1)

dfm.raw_df                      # raw CSV as a DataFrame
dfm.baseline_df                 # baseline-subtracted signal
dfm.lick_df                     # boolean lick matrix (True = feeding lick)
dfm.event_df                    # integer event matrix (run length at each
event start)
dfm.tasting_df                  # boolean tasting lick matrix
dfm.lights_df                   # per-well light state (from
OptoCol1/OptoCol2)
dfm.durations                   # dict of per-well bout duration DataFrames
dfm.intervals                   # dict of per-well inter-bout interval
DataFrames

dfm.feeding_summary()           # per-chamber feeding metrics for this DFM
dfm.plot_raw()                  # raw signal plot
dfm.plot_baselined()            # baseline-subtracted signal with threshold
lines
dfm.plot_cumulative_licks()     # cumulative lick count plot
```

---

9. Advanced analytics

All advanced analytics are available as Python functions, as GUI buttons in the Analysis Hub, and as YAML script actions. They operate on a loaded `Experiment` object.

Tidy events export

One row per feeding (or tasting) bout with columns: DFM, Chamber, Well, WellLabel, WellName, Treatment, factor columns, StartMin, Licks, Duration, AvgIntensity, MaxIntensity.

```
from pyflic import tidy_events

df = tidy_events(exp, kind="feeding")
df.to_csv("tidy_feeding_events.csv", index=False)
```

Bootstrap confidence intervals

Nonparametric percentile CIs computed by resampling at the chamber level (chambers are independent biological units). Useful for PI and other bounded/skewed metrics where parametric SEs are misleading.

```
from pyflic import bootstrap_metric

res = bootstrap_metric(exp, metric="PI", n_boot=5000, ci=0.95, seed=42)
print(res.summary)   # columns: group, n, mean, sem, ci_low, ci_high
```

Treatment comparison (ANOVA / LMM)

ANOVA (Type II) or linear mixed model (with DFM as a random intercept) comparing a metric across treatment groups, with optional Tukey HSD posthoc.

```
from pyflic import compare_treatments

res = compare_treatments(exp, metric="MedDuration", model="aov",
posthoc="tukey")
print(res.table)     # ANOVA table
print(res.posthoc)   # pairwise comparisons
```

Light-phase summary

Per-chamber feeding metrics split by light vs dark phase, derived from the `OptoCol1` column in the DFM data.

```
from pyflic import light_phase_summary

df = light_phase_summary(exp)
# columns: DFM, Chamber, Treatment, Phase (light/dark), PhaseSeconds,
Licks, Events, ...
```

Parameter sensitivity sweep

Sweep a detection parameter across a grid of values and report how Licks, Events, and MedDuration change per treatment. Each grid point re-runs the full feeding/tasting pipeline.

```
from pyflic import parameter_sensitivity

res = parameter_sensitivity(
    exp,
    parameter="feeding_event_link_gap",
    values=[0, 2, 5, 10, 15, 20],
)
print(res.grid)
# columns: feeding_event_link_gap, Group, n_chambers,
#          mean_Licks, sem_Licks, mean_Events, sem_Events,
#          mean_MedDuration, sem_MedDuration
```

Bout transition matrix (two-well)

Counts consecutive bout transitions (A-to-A, A-to-B, B-to-A, B-to-B) per chamber. Reveals whether flies alternate between wells or persist on one.

```
from pyflic import bout_transition_matrix

df = bout_transition_matrix(exp)
# columns: DFM, Chamber, Treatment, FromWell, ToWell, Count
```

Config comparison

Load two project directories and compare per-treatment metric means between them.

```
from pyflic import compare_configs

df = compare_configs(
    "project_a/", "project_b/",
    metrics=("Licks", "Events", "MedDuration"),
)
# columns: Group, Metric, mean_a, mean_b, delta, pct_change
```

YAML linting

Validate a config file programmatically:

```
from pyflic import lint_flic_config

issues = lint_flic_config("project/flic_config.yaml")
for i in issues:
    print(i.format())
```

---

10. Performance and caching

Disk cache

When `use_disk_cache=True` (the default), `load_experiment_yaml` caches the pre-computed feeding summary in `project_dir/.pyflic_cache/`. The cache key is derived from the SHA-256 hash of `flic_config.yaml` and the modification times + sizes of all DFM CSV files. If either changes, the cache is invalidated automatically.

To clear the cache manually:

```
pyflic clear-cache /path/to/project
```

Or from Python:

```
```

```
from pyflic.base import cache
cache.clear(Path("/path/to/project"))
```

Lazy loading

Pass `eager=False` to skip the feeding summary pre-computation on load. This is useful when you only need the QC viewer or want to inspect raw data:

```
exp = load_experiment_yaml("/path/to/project", eager=False)
```

---

11. Typical workflows

Workflow A -- One-click analysis with a script

1. Add a `scripts:` section to `flic_config.yaml` (see Section 5).
2. Open the hub:

   ```
   pyflic hub /path/to/project
   ```

3. Select the script from the dropdown in the **Load** group.
4. Click **Run Script**.

Workflow B -- Full analysis from scratch

```
# 1. Create the config file with the GUI
pyflic config

# 2. Validate it
pyflic lint /path/to/project

# 3. Run analysis in Python
```

```
from pyflic import load_experiment_yaml
exp = load_experiment_yaml("/path/to/project")
exp.execute_basic_analysis()
```

```
# 4. Inspect QC results interactively
pyflic qc /path/to/project
```

Workflow C -- Custom analysis in a notebook

```
from pyflic import load_experiment_yaml, bootstrap_metric,
compare_treatments

exp = load_experiment_yaml("/path/to/project")

# Explore
print(exp.summary_text())
df = exp.feeding_summary()

# Statistical comparison
res = compare_treatments(exp, metric="MedDuration", model="aov")
print(res.table)

# Bootstrap CIs on PI
boot = bootstrap_metric(exp, metric="PI", n_boot=5000)
print(boot.summary)

# Generate a full PDF report
from pyflic import write_experiment_report
write_experiment_report(exp)
```

Workflow D -- Parameter tuning

Use the QC viewer's **Params tab** to interactively adjust detection parameters and see their effect on the feeding summary without reloading data. When you find good values, update `flic_config.yaml` and re-run the analysis.

Or use the parameter sensitivity sweep to systematically evaluate a range of values:

```
from pyflic import parameter_sensitivity

res = parameter_sensitivity(
    exp,
    parameter="feeding_event_link_gap",
    values=[0, 2, 5, 10, 15, 20],
)
print(res.grid)
```

Workflow E -- QC only

```
exp = load_experiment_yaml("/path/to/project")
exp.write_qc_reports()
```

```
pyflic qc /path/to/project
```

---

12. Jupyter notebooks

The `doc/` directory contains a series of tutorial notebooks. Work through them in order for a complete introduction, or jump to the one that matches your experiment type.

`doc/01_GettingStarted.ipynb`

Start here. Covers:

- The Experiment class hierarchy and when to use each subclass
- Loading an experiment from a project directory
- How licks and events are calculated (signal processing pipeline)
- Accessing DFMs and printing experiment summaries
- Basic CLI tool overview

`doc/02_GroupedAnalysis.ipynb`

Covers analysis grouped by treatment and factor levels:

- Inspecting and filtering the feeding summary DataFrame
- Grouping results by experimental factors
- Customizing and exporting summary tables

`doc/03_ChoiceChamberAnalysis.ipynb`

For **two-well choice experiments**:

- Well-by-well metrics (preference index, simultaneous feeding)
- Two-well specific QC (bleeding check, simultaneous feeding matrix)
- Plotting utilities for choice behavior

`doc/HedonicFeeding.ipynb`

For **hedonic feeding experiments**:

- Multi-level factorial designs (e.g., concentration x genotype)
- Hedonic-specific QC and filtering
- Weighted duration summaries and specialized plots
- End-to-end example with a real dataset

`doc/ProgressiveRatio.ipynb`

For **progressive-ratio experiments**:

- Breakpoint detection and analysis

- Time-series and transition visualization
- Schedule-specific parameter recommendations

The `notebooks/` directory contains additional working examples:

- **`notebooks/load_experiment.ipynb`** -- detailed walkthrough of loading a project directory, inspecting raw DFM objects, and manually overriding parameters
- **`notebooks/run_flic_config_6h.ipynb`** -- complete 6-hour experiment pipeline: config setup, QC, full analysis, advanced plotting